

Report of *Locally Linear Embedding*

Haohao Xiao

College of Computer Science, Sichuan University
Chengdu, China

haohaoxiao.ai@gmail.com

Abstract

Locally Linear Embedding (LLE) is an unsupervised learning algorithm that computes low-dimensional, neighborhood-preserving embeddings of high-dimensional inputs while preserving local neighborhood structures. By exploiting the local symmetry of linear reconstructions, LLE maps the inputs into a single low-dimensional global coordinate system and effectively captures the global structure of nonlinear manifolds without local minima. The notations used in this paper are summarized in Appendix C.

1 Explanation of Locally Linear Embedding

In this section, we present a mathematical exposition of each component of the term Locally Linear Embedding. Through this analysis, we aim to elucidate the fundamental ideas that underlie the method.

1.1 Locally

The "Locally" highlights that reconstruction for each data point relies only on its neighbors. Mathematically, the objective function reflects this locality through two stages as follows:

- **Local Linear Reconstruction:** In the high-dimensional space, LLE ensures that the reconstruction of each data point depends solely on samples within its local neighborhood. The corresponding cost function is defined as follows:

$$\begin{aligned} \arg \min_W \sum_i \left\| \vec{X}_i - \sum_{j \in \mathcal{N}(i)} W_{ij} \vec{X}_j \right\|^2, \\ \text{s.t. } \sum_{j \in \mathcal{N}(i)} W_{ij} = 1, \end{aligned} \quad (1)$$

where $\mathcal{N}(i)$ denotes the neighborhood of $\vec{X}_i$, and W_{ij} represents the corresponding reconstruction weights. Through minimizing this objective function, LLE captures the local linear relationships in the high-dimensional space.

- **Neighborhood Preserving Mapping:** In the embedding space, LLE seeks to preserve the local linear relationships learned from the high-dimensional space.

LLE achieves this by finding the low-dimensional representations $\vec{Y}_i$ to minimize the embedding cost function as follows:

$$\begin{aligned} \arg \min_Y \sum_i \left\| \vec{Y}_i - \sum_{j \in \mathcal{N}(i)} W_{ij} \vec{Y}_j \right\|^2, \\ \text{s.t. } \sum_i \vec{Y}_i = 0, \quad \frac{1}{N} \sum_i \vec{Y}_i \vec{Y}_i^T = I, \end{aligned} \quad (2)$$

where W_{ij} are the reconstruction weights computed in the high-dimensional space. By minimizing the embedding cost function, LLE maintains the local linear geometry of each neighborhood in the embedding space.

1.2 Linear

The "Linear" emphasizes that reconstructions of the data point are confined to linear subspace spanned by its neighbors. Each data point can be reconstructed as a linear combination of its neighboring points, which can be mathematically expressed as:

$$\vec{X}_i = \sum_{j \in \mathcal{N}(i)} W_{ij} \vec{X}_j, \quad (3)$$

where $\mathcal{N}(i)$ denotes the local neighborhood of the data point $\vec{X}_i$, and W_{ij} are the reconstruction weights computed in the high-dimensional space.

1.3 Embedding

The "Embedding" refers to the process of mapping high-dimensional data points into a lower-dimensional space in a way that best preserves their intrinsic neighborhood and topological structures. Mathematically, this mapping is defined as:

$$\Phi : \mathbb{R}^D \rightarrow \mathbb{R}^d, \quad \Phi(\vec{X}_i) = \vec{Y}_i, \quad (4)$$

where $\vec{Y}_i$ denotes the low-dimensional representation of the high-dimensional point $\vec{X}_i$.

In Locally Linear Embedding (LLE), the low-dimensional representations $\vec{Y}_i$ are computed by minimizing the objective function given in Eq 2. Through this procedure, LLE constructs a mapping that preserves the local geometric structure of the data while uncovering the underlying low-dimensional manifold.

2 Assumptions of LLE

In this section, we describe the key assumptions underlying the Locally Linear Embedding (LLE) algorithm.

2.1 Local Linearity

LLE assumes that a data point and its nearest neighbors can be modeled as locally linear. Specifically, for each data point $\vec{X}_i \in \mathbb{R}^D$, it lies approximately within the linear subspace spanned by its neighbors $\mathcal{N}(i)$. Under this assumption, each data point can be linearly reconstructed from its neighbors as follows:

$$\vec{X}_i \approx \sum_{j \in \mathcal{N}(i)} W_{ij} \vec{X}_j, \quad (5)$$

where the reconstruction weights W_{ij} characterize the local geometric relationships within the neighborhood.

2.2 Smooth and Well-Sampled

The manifold is assumed to be smooth and well-sampled. More precisely, "smooth" and "well-sampled" mean that for data sampled from a d -dimensional manifold, the curvature and sampling density are such that each data point has on the order of $2d$ neighbors which define a roughly linear patch on the manifold with respect to some metric in the input space. Under such conditions, LLE can characterize the local geometry in the neighborhood of each data point by linear coefficients that reconstruct the data point from its neighbors.

2.3 Neighborhood Preservation

Suppose the data lie on or near a manifold of dimensionality $d \ll D$. To a good approximation, it can be assumed that there exists a linear mapping consisting of a translation, rotation, and rescaling that maps the high-dimensional coordinates of each neighborhood to global internal coordinates on the manifold. By design, the reconstruction weights W_{ij} reflect those geometric properties of the data that are invariant to exactly such transformations (see Appendix A for a detailed derivation). Their characterization of local geometry in the input space is equally valid for local patches on the manifold. In particular, the same weights W_{ij} that reconstruct the input $\vec{X}_i$ in D dimensions can be used to reconstruct its low-dimensional coordinates $\vec{Y}_i$ as follows:

$$\vec{Y}_i \approx \sum_{j \in \mathcal{N}(i)} W_{ij} \vec{Y}_j, \quad (6)$$

where $\mathcal{N}(i)$ denotes the set of neighbors of $\vec{X}_i$, and W_{ij} are the reconstruction weights computed in the high-dimensional space.

3 The problems to be solved by LLE

In general, LLE aims to compute a low-dimensional embedding of high dimensional data sampled from an underlying manifold with noise. It addresses nonlinear dimensionality reduction by capturing the local geometric structures

in the high-dimensional space and preserving them in the low-dimensional representation.

Formally, given high-dimensional data points $\vec{X}_1, \vec{X}_2, \dots, \vec{X}_N \in \mathbb{R}^D$, LLE aims to find their low-dimensional representations $\vec{Y}_1, \vec{Y}_2, \dots, \vec{Y}_N \in \mathbb{R}^d$ ($d \ll D$). The method preserves the local relationships among data points, which are characterized by the reconstruction weights W_{ij} computed in the high-dimensional space. The corresponding objective function is introduced in Section 1.

4 The limitations of LLE

In this section, we discuss the various limitations of LLE to clarify the conditions under which LLE is applicable.

4.1 Neighborhood Size

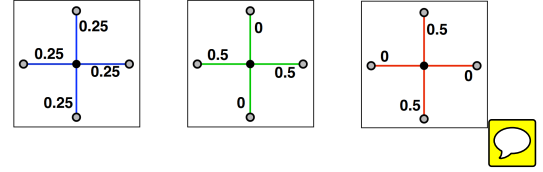


Figure 1: A two dimensional data point whose four neighbors lie at the corners of a diamond. In this case, many different settings of the weights lead to zero reconstruction error.

The algorithm can only be expected to recover embeddings whose dimensionality d is strictly less than the number of neighbors K and researchers have observed that some margin between d and K is generally necessary to obtain a topology-preserving embedding. In fact, the exact relation between K and the faithfulness of the resulting embedding remains an important open question.

If there are more neighbors K than input dimension D , then each data point can be reconstructed perfectly from its neighbors, and the local reconstruction weights are no longer uniquely defined. The specific counterexample is shown in Fig. 1.

4.2 Data Distribution

As discussed in Section 2.2, the manifold is required to be both smooth and well-sampled. If the data are unevenly distributed or sparsely sampled, the local neighborhoods may fail to capture the true geometric structure of the manifold. In such cases, the local linearity assumption of LLE breaks down, causing the algorithm to fail to produce a faithful low-dimensional mapping of the original manifold. Therefore, LLE performs reliably only when the data are sufficiently dense and uniformly distributed over the underlying manifold.

4.3 Complexity

For each data point, LLE requires computing its neighbors and solving for the local linear reconstruction weights. When the dataset is large, especially in high-dimensional spaces, the computational cost and memory requirements become substantial. In the final step, LLE needs to solve for

the bottom $d + 1$ nonzero eigenvectors of an $N \times N$ sparse matrix M , where d is the dimensionality of the embedding space. Without special optimization techniques, computing these bottom eigenvectors scales as $O(dN^2)$, increasing linearly with the number of embedding dimensions d and quadratically with the number of data points N .

4.4 Generalization Capability

LLE exhibits limited generalization capability. When new data points are introduced, LLE cannot directly compute their low-dimensional representations based on the previously computed embedding structure. Instead, it requires reapplying the entire LLE procedure to the updated dataset, including neighbor search, reconstruction weight estimation, and eigen-decomposition. This results in substantial computational overhead and makes LLE inefficient for dynamic or continuously growing datasets. Consequently, LLE is more suitable for static datasets where all samples are available beforehand, rather than for applications that demand real-time updates.

5 LLE vs PCA

In this section, we compare LLE and PCA from multiple perspectives to gain a more comprehensive understanding of their characteristics and differences.

5.1 Objective Functions

The objective function of PCA can be mathematically formulated as follows:

$$\begin{aligned} \arg \max_{\mathbf{p}_i} \mathbf{p}_i^T \mathbf{C}_X \mathbf{p}_i, \\ \text{s.t. } \mathbf{p}_i^T \mathbf{p}_i = 1, \end{aligned} \quad (7)$$

where $\mathbf{C}_X$ is the covariance matrix of $\mathbf{X}$ and $\mathbf{p}_i$ is the i -th principal component. PCA seeks a set of orthogonal directions that maximize the variance of the projected data, thus preserving as much global information as possible.

In contrast, the objective function of LLE, which has been defined in Section 1.1, aims to preserve the local neighborhood structure of data points by minimizing the reconstruction error of each point from its neighbors. This allows LLE to maintain the local geometric relationships in the low-dimensional embedding.

5.2 Motivation

The goal of PCA is to preserve as much of the variance corresponding to the signal in the data as possible during dimensionality reduction. In contrast, LLE aims to maintain the local neighborhood structure of the data while reducing its dimensionality.

5.3 Problem to be addressed

In PCA, one computes the linear projections of greatest variance from the top eigenvectors of the data covariance matrix. LLE addresses nonlinear dimensionality reduction by capturing the local geometric structures in the

high-dimensional space and preserving them in the low-dimensional representation.

5.4 Advantages of LLE over PCA

LLE is capable of capturing the nonlinear ture of data, while PCA is limited to linear relationships. As a result, LLE is particularly suitable for high-dimensional nonlinear data, as it can more effectively uncover the underlying low-dimensional manifold.

6 Experiments

In this section, we perform dimensionality reduction on high-dimensional image data from the MNIST dataset and subsequently compare the classification performance after applying LLE and PCA.

6.1 Dataset

The MNIST dataset is a widely used benchmark for handwritten digit classification. It consists of 70,000 grayscale images with a resolution of 28×28 pixels, covering 10 digit classes from 0 to 9. For this experiment, the first 2,000 images from the training set and the first 2,000 images from the test set were selected as the experimental dataset.

6.2 Dimensionality Reduction

We apply Locally Linear Embedding (LLE) to reduce the dimensionality of the training and test sets to two dimensions. The resulting 2D representations of the datasets are illustrated in Fig 2.

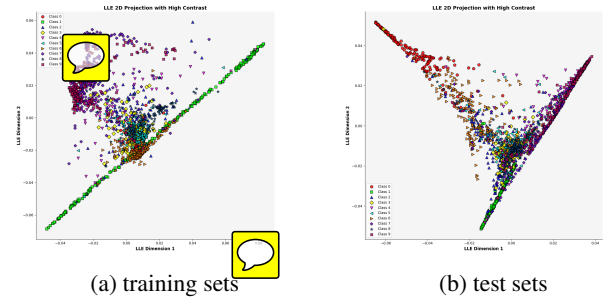


Figure 2: LLE Dimensionality Reduction Results

Below is the code of the LLE algorithm.

```
import numpy as np
from scipy.spatial import cKDTree

def LLE(X, neighbors, out_dim, delta):
    N, D = X.shape
    K = neighbors
    tree = cKDTree(X)
    dis, idxs = tree.query(X, k=K + 1)
    neighbors = idxs[:, 1:]
    W = np.zeros((N, N))
    for i in range(N):
        idx = neighbors[i]
        Z = X[idx] - X[i]
        G = Z @ Z.T
```

```

G = G + np.eye(K) * delta**2 *
    np.trace(G)/K
ones = np.ones(K)
w = np.linalg.solve(G, ones)
w = w / np.sum(w)
W[i, idx] = w
I = np.eye(N)
M = (I - W).T @ (I - W)

eigenvalues, eigenvectors = np.
    linalg.eigh(M)
sorted_indices = np.argsort(
    eigenvalues)
low_indices = sorted_indices[1:
    out_dim+1]
low_ = eigenvectors[:, low_indices]
return low_

```

As shown in Fig 2, we select the eight nearest neighbors to define the local neighborhood. From the visualization results, most classes exhibit a certain degree of clustering after dimensionality reduction. However, overlaps between classes can still be observed, particularly in the dense central region where points from different categories are intertwined. This indicates that after reducing the MNIST data to two dimensions using LLE, the separability between some classes becomes compressed.

6.3 Influence of Parameters

To examine how the parameters of LLE affect its performance, we performed classification experiments under different regularization parameters, considering multiple combinations of KNN neighbor numbers and LLE neighborhood sizes. The classification accuracy was evaluated across various embedding dimensions, and the results are presented in Fig 3 and Table 1.

Table 1: Best classification accuracy (%) for different LLE neighborhood sizes and KNN neighbor numbers.

LLE k	KNN k					
	2	4	8	16	32	
1	77.60	79.35	81.65	79.80	76.55	
2	75.25	79.55	81.80	78.85	73.20	
4	78.10	83.15	84.10	81.35	75.35	
8	77.65	82.60	84.20	81.10	75.00	
16	75.60	82.45	84.75	78.65	71.55	
32	69.85	81.85	84.15	76.00	66.25	

We selected a regularization parameter of $\Delta = 0.01$ to visualize the trends in both embedding and classification results. The experiments show that when the number of neighbors in the KNN classifier is small, classification accuracy remains relatively stable as the embedding dimension increases, exhibiting good robustness. In contrast, when the KNN neighbor number is large, accuracy fluctuates more noticeably with increasing dimensions, indicating poorer stability. In low-dimensional spaces, classification accuracy initially rises rapidly with increasing dimension, as higher dimensions capture more discriminative features. However, beyond a certain dimensional threshold, accuracy begins to

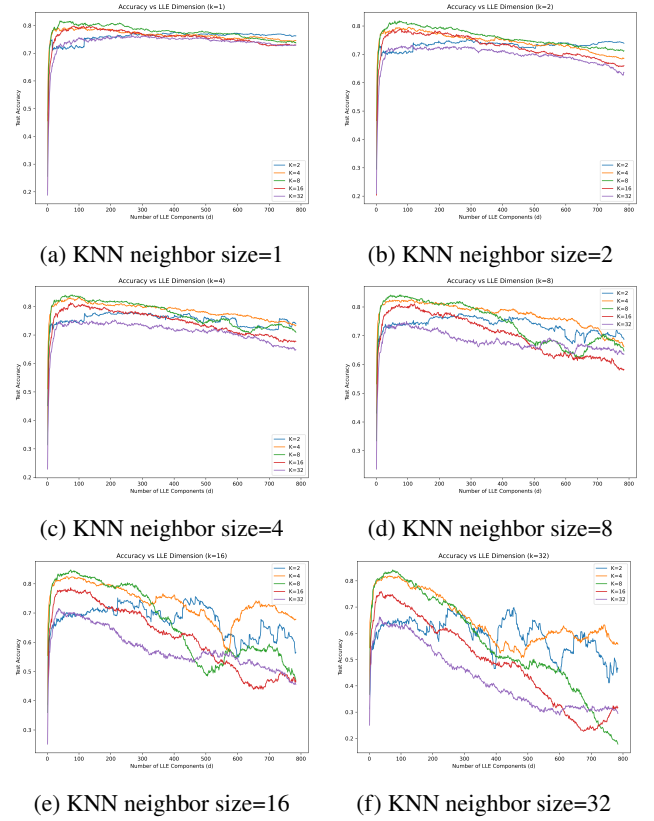


Figure 3: Impact of LLE and KNN neighborhood sizes on classification accuracy.

decrease and eventually converges to a stable value. This behavior suggests that in higher-dimensional embeddings, the inclusion of noisy dimensions weakens the representation of useful features, thereby affecting classification performance. Furthermore, the neighborhood size in LLE has a significant impact on the quality of the low-dimensional embeddings. When the neighborhood size is too small, the algorithm cannot adequately capture the local manifold structure, resulting in lower overall classification accuracy. Conversely, if the neighborhood size is too large, the local linearity assumption is weakened, which also leads to performance degradation. These observations align with theoretical expectations, indicating that the neighborhood size in LLE should be controlled within an appropriate range to achieve optimal dimensionality reduction and classification results.

6.4 Comparison

In this section, we will compare LLE with PCA on classification task and $2d$ visualization. As illustrated in Fig 2 and Fig 4, when the MNIST dataset is reduced to two dimensions, the distributions obtained by PCA and LLE exhibit clear differences. In the LLE results, samples of the same class tend to cluster along an approximately linear direction, whereas in the PCA results, they are more evenly distributed within specific regions of the plane.

According to Fig 3 and Fig 5, which illustrate the classification performance of LLE and PCA respectively, PCA maintains relatively stable accuracy when reduced to higher-dimensional spaces regardless of the parameters chosen for the KNN classifier. In contrast, the performance of LLE is

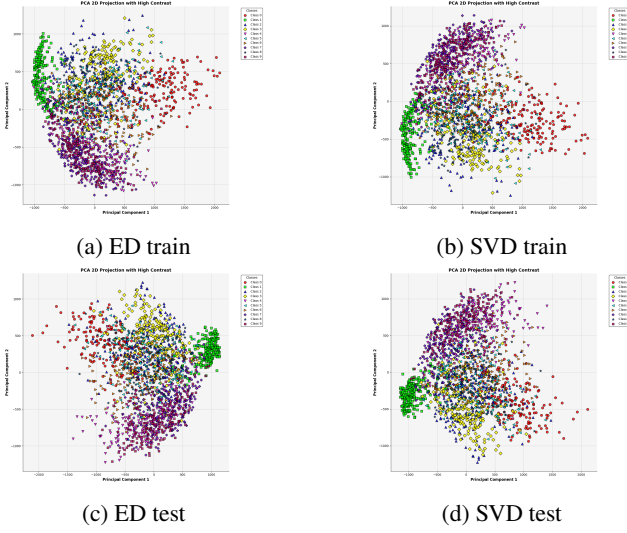


Figure 4: PCA Dimensionality Reduction Results

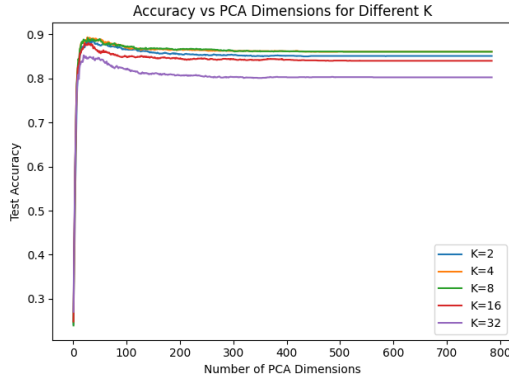


Figure 5: Recognition rate vs. dimensionality reduction

more sensitive to the number of neighbors in KNN. When this parameter becomes large, the classification accuracy of LLE in higher-dimensional embeddings fluctuates considerably, showing poorer stability.

Appendix A Invariance

The weights in LLE have three geometric invariances. We will provide proofs of these invariances in the following sub-sections.

Appendix A.1 Translation Invariance

All data points are translated by a fixed vector $\vec{v}$, such that $\vec{x}'_i = \vec{x}_i + \vec{v}$. The reconstruction error after translation can be written as follows:

$$\begin{aligned} \vec{x}'_i - \sum_j W_{ij} \vec{x}'_j &= (\vec{x}_i + \vec{v}) - \sum_j W_{ij} (\vec{x}_j + \vec{v}) \\ &= \left(\vec{x}_i - \sum_j W_{ij} \vec{x}_j \right) + \left(1 - \sum_j W_{ij} \right) \vec{v}. \end{aligned} \quad (8)$$

Since we enforce the constraint $\sum_j W_{ij} = 1$, the second term in the above equation is 0. Therefore, translation does not affect the reconstruction error, nor does it affect the optimal W_{ij} .

Appendix A.2 Rescaling Invariance

All data points are scaled by the same factor s : $x'_i = s x_i$. The reconstruction error after rescaling can be written as follows:

$$x'_i - \sum_j W_{ij} x'_j = s \left(x_i - \sum_j W_{ij} x_j \right). \quad (9)$$

The squared error becomes as follows:

$$\left\| x'_i - \sum_j W_{ij} x'_j \right\|^2 = s^2 \left\| x_i - \sum_j W_{ij} x_j \right\|^2. \quad (10)$$

Since all errors are multiplied by the same constant s^2 , the optimal solution W_{ij} is unaffected.

Appendix A.3 Rotation Invariance

Assume that all data points are rotated by an orthogonal matrix R , such that $\vec{x}'_i = R \vec{x}_i$. The reconstruction error after rotation can be expressed as follows:

$$\begin{aligned} \left\| x'_i - \sum_j W_{ij} x'_j \right\|^2 &= \left\| R x_i - \sum_j W_{ij} R x_j \right\|^2 \\ &= \left(R x_i - \sum_j W_{ij} R x_j \right)^T \left(R x_i - \sum_j W_{ij} R x_j \right) \\ &= \left(x_i - \sum_j W_{ij} x_j \right)^T R^T R \left(x_i - \sum_j W_{ij} x_j \right) \\ &= \left\| x_i - \sum_j W_{ij} x_j \right\|^2. \end{aligned} \quad (11)$$

Since R is an orthogonal matrix satisfying $R^T R = I$, the reconstruction error remains unchanged under rotation. Therefore, rotation does not affect the reconstruction error, nor does it alter the optimal weights W_{ij} .

Appendix B Minimizing the Objective Function

In this section, we explicitly derive the procedure for minimizing the two error terms in the LLE algorithm.

Appendix B.1 Reconstruction Error

The reconstruction error has been introduced in Eq 1. Each neighbor X_j of the data point X_i can be rewritten as:

$$X_j = X_i + (X_j - X_i). \quad (12)$$

Substituting this expression into Eq 1 yields:

$$\begin{aligned} \varepsilon_i &= \left\| X_i - \sum_j w_{ij} (X_i + (X_j - X_i)) \right\|^2 \\ &= \left\| X_i - \sum_j w_{ij} X_i - \sum_j w_{ij} (X_j - X_i) \right\|^2. \end{aligned} \quad (13)$$

Since LLE enforces the constraint $\sum_j W_{ij} = 1$, the first term vanishes:

$$X_i - \sum_j W_{ij} X_j = X_i \left(1 - \sum_j W_{ij} \right) = 0. \quad (14)$$

Therefore, the reconstruction error simplifies to:

$$\varepsilon_i = \left\| \sum_j W_{ij} (X_j - X_i) \right\|^2. \quad (15)$$

To simplify ε_i , define the local neighborhood matrix as:

$$Z_i = [X_{j_1} - X_i, X_{j_2} - X_i, \dots, X_{j_K} - X_i] \in \mathbb{R}^{D \times K}, \quad (16)$$

and the corresponding weight vector as:

$$w_i = [W_{ij_1}, W_{ij_2}, \dots, W_{ij_K}]^T \in \mathbb{R}^K, \quad (17)$$

where K is the number of neighbor. With these definitions, the reconstruction term can be expressed compactly as:

$$\sum_j W_{ij} (X_j - X_i) = Z_i w_i. \quad (18)$$

Hence, the reconstruction error becomes:

$$\varepsilon_i = \|Z_i w_i\|^2. \quad (19)$$

The squared Euclidean norm can be equivalently represented as an inner product:

$$\|Z_i w_i\|^2 = (Z_i w_i)^T (Z_i w_i) = w_i^T (Z_i^T Z_i) w_i. \quad (20)$$

Thus, the local reconstruction error can be expressed in a quadratic form:

$$\varepsilon_i = w_i^T G_i w_i, \quad (21)$$

where

$$G_i = Z_i^T Z_i \quad (22)$$

is the local Gram matrix.

To determine the optimal reconstruction weights w_i , LLE imposes the constraint:

$$\mathbf{1}^T w_i = 1. \quad (23)$$

Introducing the Lagrange multiplier λ , the Lagrangian function can be written as:

$$\mathcal{L}(w_i, \lambda) = w_i^T G_i w_i - \lambda (\mathbf{1}^T w_i - 1). \quad (24)$$

Taking the derivative of $\mathcal{L}$ with respect to w_i and setting it to zero gives:

$$\frac{\partial \mathcal{L}}{\partial w_i} = 2G_i w_i - \lambda \mathbf{1} = 0. \quad (25)$$

Hence, we obtain the following linear system:

$$G_i w_i = \frac{\lambda}{2} \mathbf{1}. \quad (26)$$

Solving for w_i is therefore equivalent to solving the linear system in Eq. 26. After obtaining w_i , it is normalized such that its elements sum to one  $\mathbf{1}^T w_i = 1$.

Appendix B.2 Embedding Error

The embedding cost function has been introduced in Eq. 2. It can be rewritten as:

$$Y_i - \sum_j W_{ij} Y_j = \sum_j (\delta_{ij} - W_{ij}) Y_j, \quad (27)$$

where

$$\delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases} \quad (28)$$

Then the embedding cost can be expressed as:

$$\begin{aligned} \Phi(Y) &= \sum_i \left(\sum_j (\delta_{ij} - W_{ij}) Y_j \right)^T \left(\sum_k (\delta_{ik} - W_{ik}) Y_k \right) \\ &= \sum_i \sum_{j,k} (\delta_{ij} - W_{ij})(\delta_{ik} - W_{ik}) Y_j^T Y_k \\ &= \sum_{i,j,k} Y_j^T [(\delta_{ij} - W_{ij})(\delta_{ik} - W_{ik})] Y_k. \end{aligned} \quad (29)$$

Define a matrix whose elements are given by:

$$M_{jk} = \sum_i (\delta_{ij} - W_{ij})(\delta_{ik} - W_{ik}). \quad (30)$$

Thus, the embedding cost function can be simplified as:

$$\Phi(Y) = \sum_{j,k} M_{jk} Y_j^T Y_k = \text{Tr}(Y^T M Y), \quad (31)$$

where

$$M = (I - W)^T (I - W). \quad (32)$$

In LLE, the embedding vectors are constrained to be orthonormal, i.e., $Y^T Y = I$. To enforce this constraint, we introduce a Lagrange multiplier matrix Λ :

$$\mathcal{L}(Y, \Lambda) = \text{Tr}(Y^T M Y) - \text{Tr}(\Lambda(Y^T Y - I)). \quad (33)$$

Taking the derivative of $\mathcal{L}$ with respect to Y and setting it to zero yields:

$$2MY - 2Y\Lambda = 0 \quad \Rightarrow \quad MY = Y\Lambda. \quad (34)$$

Therefore, the optimal embedding Y is formed by the eigenvectors of the matrix M .

Appendix C Notations

Table 2: Variables

Notation	Description
d	Output dimension
D	Input dimension
K	Number of nearest neighbors
N	Number of samples
W_{ij}	Reconstruction weight of the j -th data point in reconstructing the i -th data point
ϵ	Reconstruction error for a single data point
w_i	Reconstruction weight for the i -th neighbor of a data point which satisfies $\sum_i w_i = 1$
G_{jk}	Element of the local Gram matrix G , computed as $(\vec{x} - \vec{\eta}_j) \cdot (\vec{x} - \vec{\eta}_k)$ for neighbors $\vec{\eta}_j, \vec{\eta}_k$
δ_{jk}	Kronecker delta: $\delta_{jk} = 1$ if $j = k$, otherwise 0
Δ	Small regularization parameter ($\Delta^2 \ll 1$)
$\text{Tr}()$	Trace of a matrix
M_{ij}	Element of the cost matrix M , defined as $\delta_{ij} - W_{ij} - W_{ji} + \sum_k W_{ki} W_{kj}$
ρ_{ij}	Dot product of the i -th and j -th neighbor vectors
S_{ij}	Element of the squared pairwise distance matrix S
λ_n	n -th smallest eigenvalue of the cost M
z	A discrete hidden variable

Table 3: Vector

Notation	Description
$\vec{X}_i$	Input vector of the i -th data point
$\vec{x}$	High-dimensional input vector
$\vec{Y}_i$	Low-dimensional embedding vector of the i -th data point
$\vec{y}$	Low-dimensional embedding vector
$\vec{\eta}_j$	The vector of the j -th nearest neighbor of a data point $\vec{x}$
$\vec{v}$	Eigenvector
$\vec{\mu}$	High-dimensional mean vector

Table 4: Matrix

Notation	Description
G	Local Gram matrix ($K \times K$) with elements $G_{jk} = (\vec{x} - \vec{\eta}_j) \cdot (\vec{x} - \vec{\eta}_k)$
I	Identity matrix
M	The matrix ($N \times N$) for LLE embedding, defined as $(I - W)^\top (I - W)$
W	Reconstruction weight matrix ($N \times N$) for the dataset, with W_{ij} as the weight of $\vec{X}_j$ in reconstructing $\vec{X}_i$
Y	Matrix of low-dimensional embeddings ($N \times d$), where each row is $\vec{Y}_i$
Σ	Covariance matrix ($d \times d$)
ψ	Diagonal covariance matrix ($D \times D$)
Λ	A $D \times d$ loading matrix that describes the locally linear mapping from low to high dimensional observations.
γ	$\gamma_{zn} = P(z \vec{x}_n, \vec{y}_n)$
ω	$\omega_{zn} = \gamma_{zn} / \sum_{n'} \gamma_{zn'}$